Orbit Patrol

60 marks | 120-minute practice | Coding 48 + handwritten UML 12

A blue player fires upward at moving targets. Green targets give 10 points; pink targets give 20. The game stores targets and bullets in your own singly linked list. Hitting a target removes both the target and the bullet. A bullet that leaves the window is also removed.

Part	Assessed work	Marks
C1	Objects, inheritance, geometry and movement	7
C2	Node and core linked-list operations	9
C3	Traversal, target search and complexity	6
C4	Unseen algorithm: remove all inactive nodes	6
C5	Shooting and game-update order	5
C6	Selection sort on nodes	3
C7	DrawPanel, timer and GUI events	5
C8	Save AND load the game state	7
U	UML from a separate screenshot + constraints	12
	Total	60

Starting conditions

Unzip **Orbit_Patrol_NetBeans_Starter.zip** and open **OrbitPatrolMock** in NetBeans. Java 11 or newer. MainFrame and DrawPanel both include editable .form files. The starter compiles; controls and a blank canvas are expected until you complete the relevant TODOs.

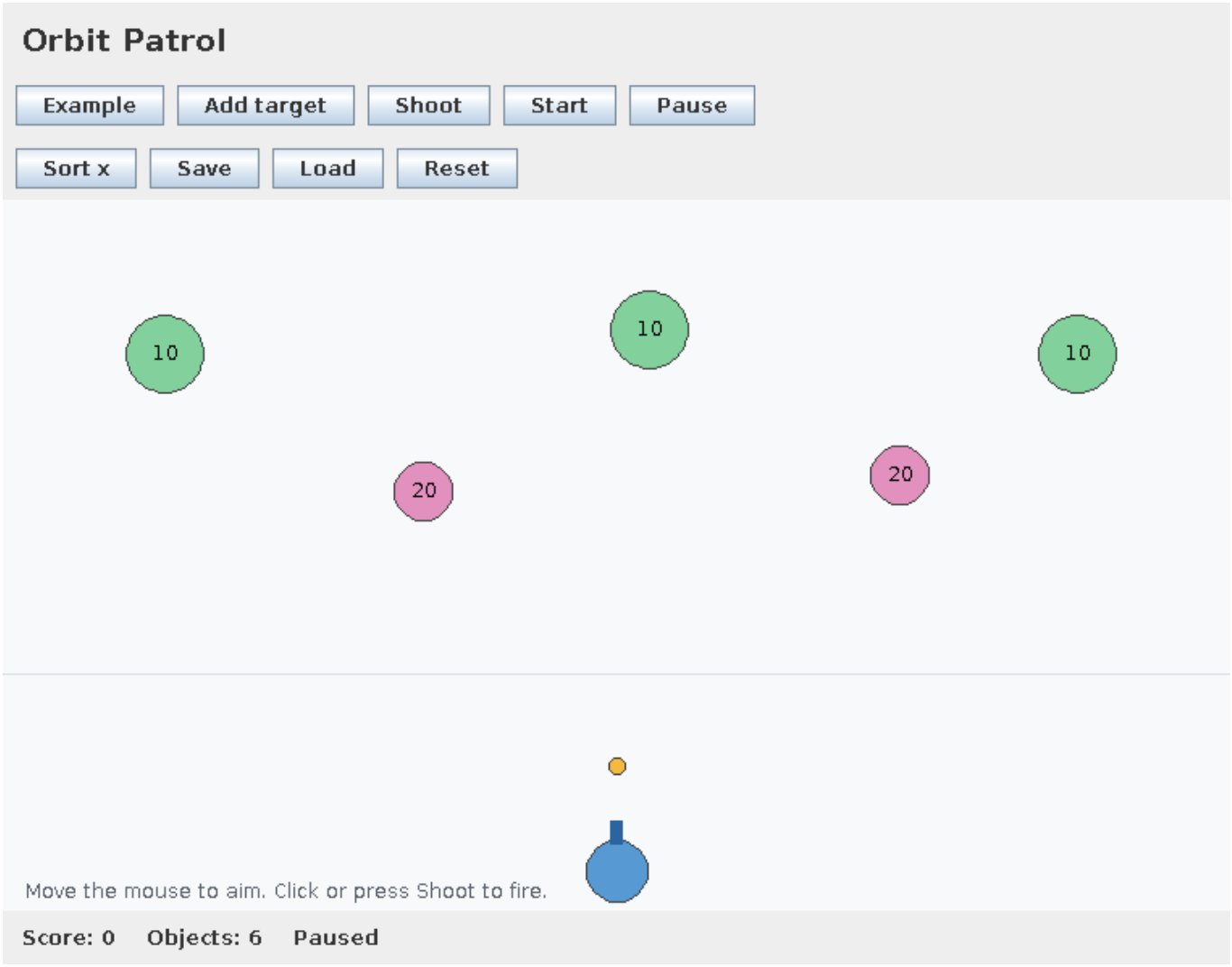
Supplied: layout, registered listeners, player aiming, example/random targets, collision coordination, file parsing, subclass file records and GUI file dialogs. Supplied code earns no marks. Complete only the labelled TODO C1-C8 regions.

Rules: use Node/getNext/setNext and a stored size; no tail field, ArrayList, built-in LinkedList, or replacement arrays for list algorithms. Parsing arrays in SnapshotCodec are allowed. Added objects are non-null. Methods must handle empty and single-node lists.

Timing: 100 minutes coding, 20 minutes UML. For a fresh modelling attempt, do pages 6-7 before opening source. Use your classroom cheat sheet; keep the separate solutions closed. This is original practice, not an official paper or a forecast of tomorrow's exact exercise.

The completed game

Reference render of the completed Java Swing components.



All circle coordinates are centre coordinates. Width and height refer to the drawing area, initially 760 by 440 pixels. The Player is stored separately; the list contains only Target and Bullet objects.

C1 - object classes: 7 marks

Orb constructor (1): store x, y, vx, vy, radius and color. Orb is abstract. Coordinates and velocities are integers; velocity is pixels per update.

Orb.draw(Graphics g) (1): draw a filled circle with the stored colour and a dark outline. Use the radius correctly around the centre.

Orb.collidesWith(Orb other) (1): return true when the distance between centres is at most the sum of radii; touching counts. Compare squared distances if preferred.

Target constructor (1): retain the supplied super(...) call and store points. Target.draw is supplied and reuses the superclass drawing method.

Target.move(width, height) (2): first apply vx/vy. Keep the complete target inside the upper two-thirds of the drawing area. When it crosses a boundary, clamp its centre to the valid boundary and reverse the corresponding velocity. Check horizontal and vertical boundaries independently.

Bullet.move(width, height) (1): apply vx/vy. Mark it inactive only after the complete circle is outside any edge. Do not change links inside an object's move method.

Nodes and traversal

C2 - Node and MyList: 9 marks

Method / class	Contract	Marks
Node	Constructor + getData/setData + getNext/setNext. Data type is Orb; successor field is named node.	2
sizeConstant(); clear()	Read stored size in O(1). Clear sets root to null and size to zero.	1
addToHead(Orb data)	Add one new node before the current root.	1
addToTail(Orb data)	Append one new node; empty-list case included. No stored tail field.	2
delete(int index)	Remove exactly the indexed node. Bounds: 0 <= index < size; otherwise throw IndexOutOfBoundsException before mutation.	3

After every structural change, stored size equals the number of reachable nodes. The final successor is null. Increment/decrement size exactly once per addition/removal.

C3 - using objects through nodes: 6 marks

MyList.move(width, height) (1): traverse directly and call move on every stored object. The runtime object determines which overridden move method executes.

MyList.draw(Graphics g) (1): traverse directly and draw every active object. Skip objects marked inactive.

findTargetHitBy(Bullet bullet) (3): return the first active Target in list order that collides with the supplied active bullet. Ignore other bullets and inactive targets. Return null when none qualifies. Use instanceof and the appropriate cast.

Complexity (1): add comments giving worst-case time for move and findTargetHitBy, and explain why repeated get(index) calls would be a poor way to visit every node.

A checkpoint before continuing

With C1-C3 and C7 complete, Example should show five targets and one bullet. Start should move them; Pause should stop updates. Aiming changes the separate player's x coordinate. Hits/cleanup require C4-C5.

The main algorithm

C4 - removeInactive(): 6 marks

Collisions and off-screen movement mark objects inactive. Implement **int removeInactive()** in MyList. Remove every node whose stored Orb is inactive and return the number of nodes removed.

Make one traversal in O(n) time and O(1) auxiliary space. Surviving objects and nodes must be the same instances in the same relative order. Do not rebuild the list, copy to another collection, or repeatedly call delete(index). Keep size correct. Include a short complexity comment.

Input flags in list order	Survivors	Return value
A, X, X, A	A, A	2
X, X, A	A	2
A, X, X	A	2
X, X, X	empty	3
empty	empty	0

A = active, X = inactive. Test a single active node and a single inactive node too.

Marking: correct removal including head/consecutive cases (4); correct size, return value and preserved survivor order/identity (1); required complexity and explanation (1).

C5 - shooting and update order: 5 marks

Game.shoot() (2): create and append one Bullet at player x and **player y - player radius - 18**. Set vx = 0, vy = -8 and radius = 5. The separate Player is not added to the list.

Game.move() (3): each update must first move the objects, then call the supplied resolveHits(), then remove inactive nodes. Use the current width/height. This lets collisions mark objects before structural deletion.

Read the supplied resolveHits: it checks each active bullet, calls your findTargetHitBy, marks the target and bullet inactive, and adds the target's points. An inactive target must not be awarded twice in the same update.

C6 - selection sort: 3 marks

Implement **MyList.sortByX()** in ascending x order using direct node references and selection sort. Find the smallest x in the remaining suffix, then swap entire Orb references using getData/setData. Do not swap only x values. Empty and one-node lists must work. Add the worst-case time complexity in a comment.

GUI and persistence

C7 - DrawPanel and MainFrame: 5 marks

DrawPanel.paintComponent(Graphics g) (2): clear through the superclass implementation, then, if game is non-null, update its size from the panel and draw it. Painting must not advance the game.

MainFrame constructor (2): replace the placeholder with a javax.swing.Timer using a 20 ms delay. On each event update the game and call the supplied refresh(). Initialise the timer but leave it stopped. Start/Pause handlers are supplied.

Shoot and mouse-moved handlers (1): Shoot calls the model then refreshes. Mouse movement passes evt.getX() to Game.aimAt then refreshes. Listener registration is supplied; a mouse press delegates to the Shoot button.

C8 - save and load: 7 marks

Implement Game.saveToFile(String fileName) and Game.loadFromFile(String fileName). Both declare throws IOException. Save/Load buttons and their try/catch dialogs are supplied. File operations pause the game and loading leaves it paused.

```
STATE;0;380
TARGET;100;95;2;1;24;10
BULLET;380;350;0;-8;5
```

First line: STATE;score;playerX. Each later nonblank line is one active object, in list order. TARGET records contain x;y;vx;vy;radius;points. BULLET records contain x;y;vx;vy;radius.

Save (3): use PrintWriter; write the STATE line; traverse nodes and write each active object's supplied toFileRecord(); close the writer. The Player is represented only by the STATE line.

Load (4): use Scanner; read the STATE line with supplied SnapshotCodec.parseState, which returns int[] {score, playerX}. Read all subsequent nonblank lines with supplied SnapshotCodec.parseOrb, which returns a new Target or Bullet. Append to a fresh MyList; replace the old list and restore score/player position. Close the scanner.

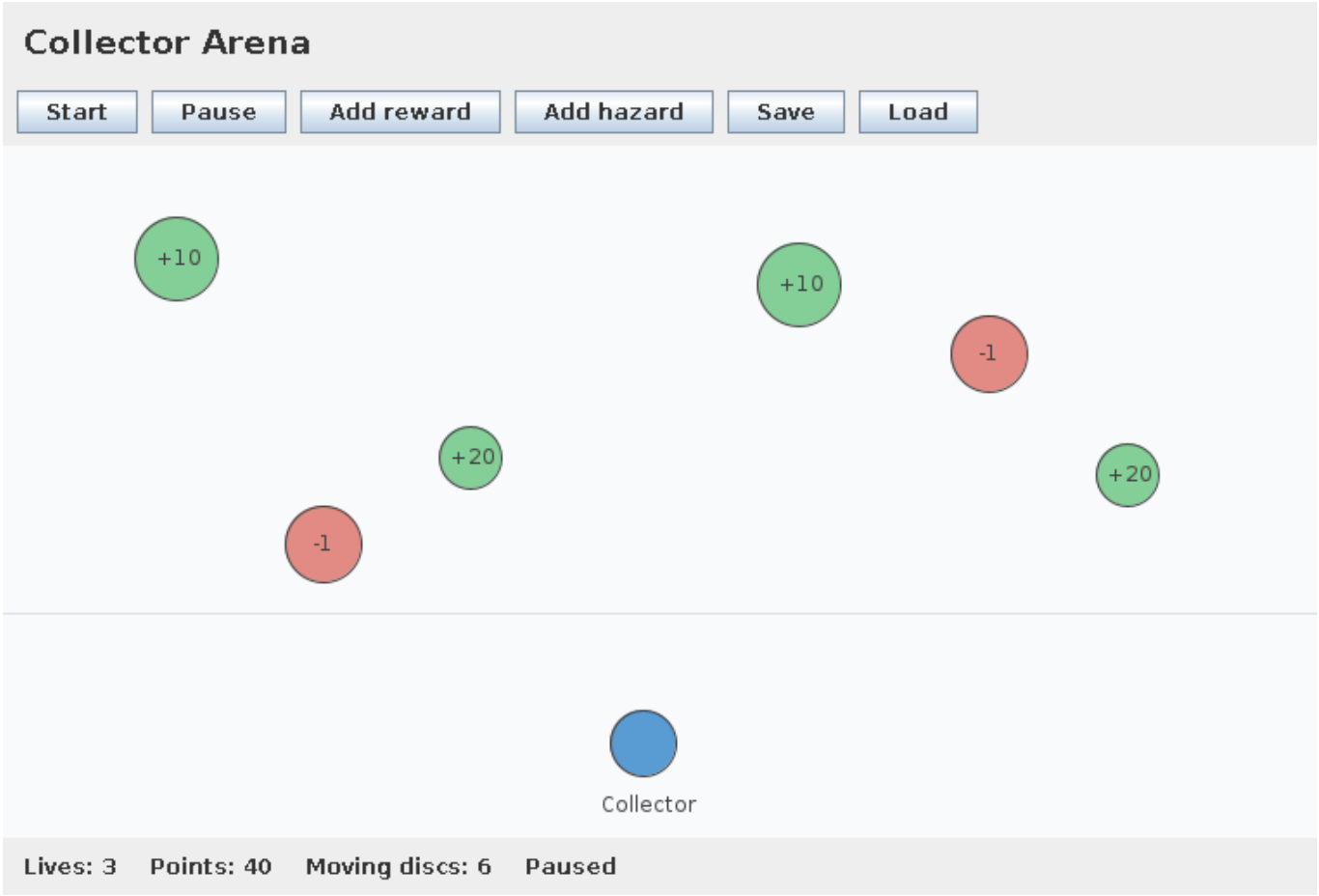
Generated files are the assessed input; malformed-file recovery is not graded. Keep the list size correct through normal add methods. Restoring velocities and subclass types is required; restoring only positions is incomplete.

Final behaviour check

Example -> Start -> aim and shoot -> Pause -> Save -> Reset -> Load. The same score, player x, object order, subclasses, positions and velocities must return. Loading twice must not duplicate objects. Start resumes movement. Check removal of consecutive inactive objects separately.

Paper UML - 12 marks

A separate application: Collector Arena. Work from this view and the constraints on the next page. Do not inspect the solutions.



The blue collector moves along the bottom. Green rewards increase points; red hazards reduce lives. The buttons add either kind of moving disc and save/load the arena. Your task is to model its object logic, not reproduce the interface layout.

What to draw

Draw one class diagram by hand. Choose meaningful class names and include necessary attributes, constructors and methods with types, parameters and visibility. Use your teacher's inheritance, filled-diamond and hollow-diamond notation. No multiplicities are required.

MainFrame, DrawPanel, Swing widgets and Timer must be omitted. The image is a Java Swing reference scene for this original exercise. It is separate from Orbit Patrol, although the programming patterns are related.

UML constraints

Use these constraints together with the Collector Arena view. Names may differ if their responsibilities and relationships are correct.

1. Every disc stores its centre x/y, radius, colour, vx/vy and an inactive flag. Use int for geometry/velocity, Color for colour and boolean for the flag. Shared geometry is directly usable by subclasses. The common Disc class is abstract; it provides draw(Graphics g), collidesWith(Disc other): boolean and an abstract move(width: int, height: int): void.

2. The collector, rewards and hazards are specialised discs. Their move methods differ: the collector moves along the bottom, rewards drift horizontally and hazards vertically. The collector owns its lives and points counters and provides loseLife(): void and addPoints(amount: int): void. A reward has an int points value; a hazard has an int damage value.

3. Arena creates and owns one collector and one DiscList. It stores width and height and coordinates move(), draw(g), addReward(), addHazard(), saveToFile(fileName) and loadFromFile(fileName). Drawing takes Graphics; file methods take String; these arena methods return void. Start/Pause are GUI responsibilities.

4. DiscList stores rewards/hazards in its own singly linked nodes, with private root: Node and size: int. It creates and owns those nodes. Include addToTail(data: Disc), move(width, height), draw(g), removeInactive(): int and sizeConstant(): int. The first three return void.

5. A Node wraps a Disc reference supplied by Arena and a successor Node reference. The wrapped Disc is not created or exclusively owned by that Node. Show its constructor and getData/setData/getNext/setNext. Name the successor field node. Include the successor attribute; you need not draw a self-loop.

6. Avoid repeating inherited geometry in each subclass. Public operations use +; private state uses -; geometry shared with subclasses uses #. Mark abstract class/method names in italics, or clearly write {abstract} if handwriting makes italics unclear.

Criterion	Marks
Appropriate classes, shared/subclass attributes and types	4
Constructors/methods, parameters and return types	3
Inheritance correctly directed towards Disc	2
Exclusive ownership and shared-reference diamonds correctly placed	2
Visibility and abstract notation; clear layout	1

A constructor has the class name and parameters but no return type. Put a diamond at the class that holds/owns the other object. A triangle points towards the superclass.

Think one step ahead

Optional practice after the timed attempt. These are plausible ways to vary the same skills, not probability estimates or predictions.

Possible project	How nodes would be used	What transfers
Shooter / moving targets	Store targets and bullets; flag hits; remove nodes.	Traversal, subclasses, collision, cleanup.
Catching falling objects	Store items with different rewards or penalties.	Override move; remove missed/caught items.
Snake / moving chain	Store ordered segments; grow at the head and remove the tail.	Head/tail handling; order and position updates.
Queue / race simulation	Store participants/jobs; update and remove finished items.	Search, sorting, extraction and timers.

Four useful variations

A - closest active target (5-10 minutes): write Target findClosestTarget(int x, int y). Return null if none exists. Compare squared distance, ignore bullets/inactive targets, keep the first on ties. Use one traversal.

B - remove by a different condition (5 minutes): adapt removeInactive into removeTargetsWorth(int points). Remove every matching Target, preserving all other objects. Test consecutive head removals.

C - reverse drawing order (5-10 minutes): reverse the node chain while retaining the same objects and size. Think about root, previous, tmp and the saved next reference. The timer/drawing traversal still works afterwards.

D - timer-spawned objects (5 minutes): add a Game update counter. Every 50 update callbacks, append a new target. Save/load the counter as well if continuation of the spawn cycle is part of the requirement.

What deserves your attention tonight

First get one working path: objects -> nodes -> movement -> timer -> repaint. Then concentrate on removing several nodes without skipping one, reconstructing saved subclasses, and sorting through direct nodes. Use extra variations only after the core mock works.

The official syllabus you supplied covers inheritance/polymorphism, selection sort, search, complexity and singly linked operations. Timer/game mechanics here are preparation for the project style you described; the UML example does not establish that tomorrow's coding exercise will be a game.